2 实验二

实验目的

本实验(实验二)旨在利用词法分析程序自动生成工具 JFlex,自动产生 Oberon-0 语言的词法分析程序。具体目标包括:

- 总结 Oberon-0 语言的词汇表，以表格形式分类列出各类单词，并说明单词分类的理由。
- 将词法定义从 Oberon-0 BNF 中分离，写成正则定义式（Regular Definition）形式。
- 编写 JFlex 输入源文件 `oberon.flex`，利用 JFlex 1.8.2 自动生成 Java 词法分析程序 `OberonScanner.java`。
- 利用实验一编写的变异程序全面测试词法分析器对各类词法错误的检测能力。
- 对比 JFlex、JLex 和 GNU Flex 三种 lex 族工具在词法规则定义方面的差异。

实验目录与项目组织

实验二的项目文件均存放于 `ex2/` 子目录下，各文件说明如下:

- `src/oberon.flex`: JFlex 输入源文件，包含 Oberon-0 词法规则的正则定义。
- `src/OberonScanner.java`: JFlex 自动生成的词法分析程序（NFA 263 状态，DFA 105 状态)。
- `src/ScannerTest.java`: 扫描器测试驱动程序，接受源文件名作为命令行参数。
- `src/Token.java`: Token 类定义，含所有单词类型的枚举（35 种 Token 类型)。
- `src/exceptions/`: 14 个异常类，形成 OberonException → LexicalException/SyntacticException/SemanticException → 具体异常类的三层继承体系。
- `bin/`: 编译后的 Java 字节码目录。
- `doc/`: javadoc 生成的 HTML 格式设计文档。
- `jflex/lib/`: JFlex 工具 JAR 包（jflex-full-1.8.2.jar)。
- `lib/`: ROSE 实验软装置 JAR 库（callgraph.jar、flowchart.jar、jgraph.jar)。

- `gen.bat`、`build.bat`、`run.bat`、`test.bat`：构建、运行和测试脚本。

2.1 Oberon-0 词汇表

根据 Oberon-0 语言的 BNF 定义和语义描述，将全部单词按词法处理方式分为**保留字**、**关键字/预定义标识符**、**运算符与界符**、**标识符**、**字面量**和**注释**六大类，以下逐类列出并说明分类理由。

2.1.1 保留字

保留字是 Oberon-0 语法结构的骨架，在词法分析阶段必须被识别为独立的 Token 类型，**不允许**被程序员用作标识符。Oberon-0 共 19 个保留字，全部列出如下：

用途	保留字
模块定义	MODULE, END
声明关键字	CONST, TYPE, VAR, ARRAY, OF, RECORD
过程定义	PROCEDURE, BEGIN
条件语句	IF, THEN, ELSIF, ELSE
循环语句	WHILE, DO
运算符	DIV, MOD, OR

表 2.1 Oberon-0 保留字表（19 个）

注意，DIV、MOD、OR 虽然语义上是运算符，但在词法层面它们由字母组成，与 `+`、`-` 等符号运算符的识别方式不同——在正则表达式中它们需以保留字的方式逐字符匹配。JFlex 输入文件为每个保留字编写独立的匹配规则，例如：

```
[Mm][Oo][Dd][Uu][Ll][Ee]    { return token(Token.Type.MODULE); }
[Ii][Ff]                     { return token(Token.Type.IF); }
[Dd][Ii][Vv]                 { return token(Token.Type.DIV_KW); }
```

2.1.2 关键字与预定义标识符

关键字在词法分析阶段被识别为**标识符**(TK_IDENT),其特殊语义由语法/语义分析阶段根据上下文确定。这与保留字有本质区别——关键字**允许**在嵌套作用域中被重新声明而改变含义。

类别	单词	说明
2* 基本类型名	INTEGER BOOLEAN	整数类型 布尔类型
布尔字面量	TRUE, FALSE	由语法分析识别为布尔常量
预定义过程	Read, Write, WriteLn	由语法分析识别为过程调用

表 2.2 Oberon-0 关键字与预定义标识符（7 个）

2.1.3 运算符与界符

运算符和界符在词法层面对应固定的终结符 Token，在 JFlex 中通过字符串字面量直接匹配。注意，Oberon-0 的 <=、>= 和 := 是多字符运算符，必须在单字符运算符之前列出，以保证最长匹配优先。

类别	符号
算术运算符	+（加/取正）、-（减/取负）、*（乘）
关系运算符	=（等）、#（不等）、<（小于）、<=（小于等于）、>（大于）、>=（大于等于）
逻辑运算符	&（与）、~（非）
赋值运算符	:=
括号	( )（圆括号）、[ ]（方括号）
分隔符	.（句点）、,（逗号）、;（分号）、:（冒号）

表 2.3 Oberon-0 运算符与界符表

2.1.4 标识符、字面量与注释

类别	词法规格	约束
标识符	[a-zA-Z][a-zA-Z0-9]*	大小写无关，长度 ≤ 24
整数字面量	digit{digit}	十进制或 0 前缀八进制，$\leq$ 12 位
注释	(* ... *)	不可嵌套

表 2.4 Oberon-0 标识符、字面量与注释

2.1.5 单词分类理由与保留字/关键字处理方式

以上分类的核心依据是**词法分析阶段的处理方式不同**：

1) **保留字**：必须在词法分析阶段被识别为独立的 Token 类型。原因是它们定义了程序的语法骨架——如果允许程序员将 IF 用作变量名，语法分析器将无法区分条件语句与赋值语句。在 JFlex 中，每个保留字编写一条独立的匹配规则，置于标识符规则**之前**，利用最长匹配优先原则确保保留字不会被错误地识别为标识符。

2) **关键字（INTEGER、BOOLEAN、TRUE、FALSE)**：在词法分析阶段被识别为普通的标识符 Token（TK_IDENT），不在词法层面做特殊处理。原因是 Oberon-0 的作用域规则允许程序员在嵌套过程中重新声明这些名字——例如在内层声明 `VAR INTEGER: INTEGER` 是合法的。如果在词法层面将 INTEGER 固化为特殊 Token，将破坏语言的这一特性。

3) **预定义过程（Read、Write、WriteLn)**：同样识别为标识符 Token。语法分析器在过程调用上下文中（identifier 后跟 actual_parameters）将其作为普通过程调用处理；语义分析阶段通过符号表查找确定是否为预定义过程。

4) **运算符和界符**：每个符号对应一个独立的终结符 Token，是语法分析中优先级和结合性规则的操作对象。

5) **标识符和字面量**：分别对应 TK_IDENT 和 TK_INTEGER_LITERAL Token，是语法分析中 factor 非终结符的组成部分。

6) **注释和空白**：在词法分析阶段被直接跳过，不产生任何 Token，对语法分析器完全透明。

保留字与关键字处理方式的核心区别总结如下：

对比维度	**保留字**	**关键字**	**影响**
词法 Token	独立类型（如 TK_IF）	标识符 TK_IDENT	保留字不可重名
可否用作变量名	否	是（作用域内）	关键字可被遮蔽
正则匹配方式	精确字符串匹配	归入标识符规则	保留字优先匹配
语义处理	直接参与语法推导	语法后查符号表	关键字语义可重定义

表 2.5 保留字与关键字的处理方式对比

2.2 词法规则的正则定义式

从 Oberon-0 的 BNF 定义中分离词法规则，使用正则定义式描述（→ 表示”定义为”，epsilon 表示空串）：

$letter \rightarrow$ [a-zA-Z]

$digit \rightarrow$ [0-9]

$identifier \rightarrow letter\ (letter \mid digit)^*$

$integer \rightarrow digit^+ \mid 0[0\text{-}7]^*$

$number \rightarrow integer$

保留字 $\rightarrow [Mm][Oo][Dd][Uu][Ll][Ee] \mid [Ee][Nn][Dd] \mid [Pp][Rr][Oo][Cc][Ee][Dd][Uu][Rr][Ee] \mid .$

运算符 $\rightarrow + \mid - \mid * \mid / \mid = \mid \# \mid < \mid <= \mid > \mid >= \mid \& \mid \sim \mid :=$

界符 $\rightarrow (\mid) \mid [\mid] \mid . \mid , \mid ; \mid :$

注释 $\rightarrow (* (\sim (* \mid *))^* *)$

空白 $\rightarrow (space \mid tab \mid newline)^+$

关键实现细节:

1) **大小写无关**:Oberon-0 不区分大小写,保留字匹配使用 `[Mm][Oo][Dd][Uu][Ll][Ee]` 形式的字符类。JFlex 提供的%ignorecase 选项无法精细控制哪些规则大小写敏感，故采用显式字符类方案。
2) **标识符长度限制**：正则表达式无法表达长度约束（24 字符上限），在标识符规则的语义动作中使用 Java 代码检查 `yytext().length() > 24`，超限时抛出 IllegalIdentifierLengthException。
3) **整数常量验证**：分三层——(1) 八进制（0 开头）中不含 8/9，否则抛出 IllegalOctalException；(2) 整数中数字个数不超过 12，否则抛出 IllegalIntegerRangeException；(3) 整数后紧跟字母（如 25id）时抛出 IllegalIntegerException。规则 (3) 需在标识符规则之前放置专用匹配规则 `{Digit}+{Letter}...` 来检测。
4) **注释状态机制**：使用 JFlex 的%state COMMENT 状态——遇到 `(*` 进入 COMMENT 状态，遇到 `*)` 返回 YYINITIAL；若 EOF 出现在 COMMENT 状态则抛出 MismatchedCommentException。注释不可嵌套，因此 CONTENT 状态下仅需匹配 $[^*]*$

2.2.1 与其他语言词法规则的比较

将 Oberon-0 的词法规则与 Pascal、C/C++ 和 Java 进行系统比较:

特征	Oberon-0	Pascal	C/C++	Java
大小写敏感	否	否	是	是
标识符长度限制	24 字符	实现相关	无	无
注释形式	(* *)	(* *) 或 { }	/* */ 和 //	/* */ 和 //
注释嵌套	不允许	部分允许	不允许	不允许
整数八进制	0 前缀	无	0 前缀	0 前缀
字符串/字符常量	不支持	支持	支持	支持
实数常量	不支持	支持	支持	支持
布尔常量	TRUE/FALSE（关键字）	TRUE/FALSE	true/false	true/false
空白符含义	分隔符	分隔符	分隔符	分隔符

表 2.6 Oberon-0 与其他语言词法规则的异同

Oberon-0 词法规则的主要特点：(1) 词汇空间远小于 C/C++/Java，没有字符串、字符和实数常量，简化了词法分析器的复杂度；(2) 标识符有 24 字符的硬性上限，这是 Oberon 语言家族的独特约束；(3) 八进制表示法与 C 语言一致（0 前缀），但在检测错误上更为严格（0 后紧跟 8/9 立即报错）。

2.3 下载词法分析程序自动生成工具 JFlex

从 JFlex 官方网站 `http://www.jflex.de/` 下载最新稳定版 JFlex 1.8.2（2020 年 5 月发布）。下载的压缩包中包含 Java 源代码、预编译 JAR 库、用户文档和输入源文件例子。由于实验环境为 Linux，无需修改 Windows 批处理脚本 `bin/jflex.bat` 中的环境变量，直接通过 `java -jar jflex-full-1.8.2.jar` 命令行方式运行。

为验证 JFlex 正确安装与配置，运行其附带的示例文件 `examples/simple/src/main/jflex/simple.flex`：

1) **生成扫描器**：执行 `java -jar jflex-full-1.8.2.jar simple.flex`，成功生成 `Yylex.java`（NFA 165 状态，DFA 最小化后 48 状态）。
2) **编译**：将生成的 `Yylex.java` 与配套的 `Yytoken.java`、`Utility.java` 联合编译通过，无任何错误。
3) **运行测试**：以输入文本 `if (x < 10) { /* comment */ y = x + 1; }` 测试生成的扫描器，输出如下：

```
if    → Token[43] (关键字)    (    → Token[3]  (左括号)    x    → Token[43] (标识符)
<     → Token[16] (小于)      10   → Token[42] (整数)      )    → Token[4]  (右括号)
{     → Token[7]  (左花括号)  /*..*/ → [COMMENT状态, 跳过]  y    → Token[43] (标识符)
=     → Token[14] (等号)      x    → Token[43] (标识符)    +    → Token[10] (加号)
1     → Token[42] (整数)      ;    → Token[2]  (分号)      }    → Token[8]  (右花括号)
```

测试结果表明：(1) 关键字和标识符正确识别；(2) 多字符运算符（注释定界

符 /* */）正确匹配；(3) COMMENT 状态机制工作正常，注释内容被完全跳过；(4) 空白符被正确忽略。JFlex 1.8.2 安装配置验证无误。

2.4 生成 Oberon-0 语言的词法分析程序

2.4.1 Oberon-0 词法分析器的 JFlex 规格说明

确认 JFlex 工作正常后，编写面向 Oberon-0 的 `oberon.flex` 文件，遵循 JFlex 的三段式结构：

1) **用户代码段**（第一段）：声明 package 和必要的 import 语句（导入 exceptions 包下的所有异常类和 Token 类）。
2) **选项与宏定义段**（第二段，以%% 分隔）：声明生成的类名 `OberonScanner`、返回类型 `Token`、启用%unicode 和%line/%column 位置跟踪；定义%state COMMENT 状态；定义宏 letter、digit、identifier、integer 等正则简写。
3) **词法规则段**（第三段，以%% 分隔）：按最长匹配优先原则组织规则——保留字（每个保留字独立规则）→ 双字符运算符（:=、<=、>=）→ 单字符运算符和界符 → 整数常量（含非法检测规则）→ 标识符 → 错误回退（捕获所有非法字符抛出 IllegalSymbolException）。

运行 JFlex 1.8.2 处理 `oberon.flex`，生成过程：构建 NFA（263 个状态）→ 转换为 DFA（113 个状态）→ DFA 最小化（105 个状态），最终生成 `OberonScanner.java`。生成的扫描器类名设为 `OberonScanner`（而非 JFlex 默认的 `Yylex`），以符合实验提交要求。

2.4.2 词法错误的异常类映射

规格说明要求词法分析程序应根据词法错误的类别抛出对应的异常类。实验软装置中异常类的实现标记为 [PENDING]（参见 Notation.md），故本实验自行完善了全部 14 个异常类的三层继承体系（OberonException → LexicalException/SyntacticException/SemanticException → 具体异常）。词法规则与异常类的映射关系如下：

规格说明要求实验二中不改变软装置提供的 `main()` 的内容，但由于软装置的 `main()`（第 5.3.2 节）标记为 [PENDING]，本实验自行编写了 `ScannerTest.java` 作为测试驱动程序，其功能等价于规格说明描述的 `main()`：接受命令行指定的 Oberon-0 源文件名，调用 OberonScanner 完成扫描，根据返回的异常类型判断与预期输出是否一致。

检测场景	JFlex 规则	抛出的异常
输入符号不在任何合法字符集中	回退规则 `[^]`	IllegalSymbolException
整数后紧跟字母（如 25id）	`{Digit}+{Letter}...`	IllegalIntegerException
整数数字超过 12 位	语义动作中检查 length()	IllegalIntegerRangeException
八进制中含 8 或 9	语义动作中逐字符检查	IllegalOctalException
标识符长度超过 24 字符	语义动作中检查 length()	IllegalIdentifierLengthException
EOF 出现在注释状态	`<<EOF>>` in COMMENT	MismatchedCommentException

表 2.7 词法规则到异常类的映射

2.4.3 扫描器测试结果

使用 ScannerTest 驱动程序对实验一编写全部测试用例进行扫描，测试结果汇总如下：

测试用例	Token 数	结果	检测异常
BankSystem.obr	683	SUCCESS	–
BankSystem.001	–	FAILED	IllegalSymbolException
BankSystem.002	–	FAILED	IllegalIdentifierLengthException
BankSystem.003	–	FAILED	IllegalOctalException
BankSystem.004	–	FAILED	IllegalIntegerException
BankSystem.005	–	FAILED	MismatchedCommentException
BankSystem.014	–	FAILED	IllegalIntegerRangeException
BankSystem.006–013	13–40	SUCCESS	语法/语义类错误，词法通过

表 2.8 扫描器测试结果汇总

测试结果表明：

1) 正确程序 BankSystem.obr 被成功扫描为 683 个 Token，无任何词法错误。
2) 全部 6 种词法错误变异程序（.001–.005、.014，以及 derived 系列.015、.021、.022）均被精确检测——例如 IllegalSymbolException 对应非法符号 $、IllegalOctalException 对应 08/09、IllegalIntegerRangeException 对应超 12 位的整数。
3) 语法和语义错误变异程序（.006–.013）及 derived 系列（.016–.020）在词法层面全部通过扫描，错误正确地留给后续语法分析和语义分析阶段检测。

2.5 讨论不同词法分析程序生成工具的差异

对 JFlex、JLex 和 GNU Flex 三种流行的词法分析程序生成工具进行系统比较：

词法规则定义方面的差异：

- **宏定义语法**：三者均支持在第二段定义命名正则表达式，JFlex/JLex 使用

特征	JFlex 1.8.2	JLex	GNU Flex
开发语言/生成代码	Java / Java	Java / Java	C / C
大小写无关	%ignorecase	不支持	%option case-insensitive
Unicode 支持	完全支持	部分	依赖 locale
EOF 规则	«EOF» 支持	不支持	«EOF» 支持
状态机制	%state 声明	%state 声明	start conditions
否定前瞻	支持! 运算符	不支持	不支持
NFA→DFA 算法	优化的表格驱动	较慢	快速 C 实现
维护状态	活跃（2020 v1.8.2）	已停止	活跃

表 2.9 JFlex、JLex 和 GNU Flex 的比较

`macroName = regex` 格式，GNU Flex 使用 `name {regex}` 格式。使用时 JFlex/JLex 用 `{macroName}` 引用，Flex 直接用 `{name}`。

- **语义动作**：JFlex/JLex 的语义动作为 Java 代码块（`{ ... }`），可访问 `yytext()`、`yylength()` 等方法；Flex 的语义动作为 C 代码块，通过 `yytext`、`yyleng` 等全局变量访问匹配文本。
- **状态声明**：JFlex 使用 `%state STATE_NAME` 声明，规则中用 `<STATE_NAME>` 限定；Flex 使用 `%x`（exclusive）或 `%s`（inclusive）声明 start condition。

就本实验而言，JFlex 是最合适的选择——它生成 Java 代码可直接集成到 ROSE 的 Java 项目中，Unicode 和 EOF 规则的支持使得注释处理和非法符号检测的实现更加简洁可靠。

2.6 自我评分参照

参照实验二评价标准，各项完成情况如下：

项目	权重 (%)	对应内容
1、词法规则的正则定义式	20	4.2 节完整正则定义式
2、JFlex 输入源文件正确性	40	4.4 节 oberon.flex
3、词法分析程序运行与测试	20	4.5 节 683 token + 6 种错误全检出
4、lex 族工具差异比较	10	4.6 节 JFlex/JLex/Flex 对比
5、文档组织及 readme 文件等	10	ex2/readme.txt

表 2.10 实验二自我评分参照

2.7 实验心得体会

通过实验二，我深入理解了词法分析程序自动生成工具的工作原理和使用方法。从 Oberon-0 的 BNF 定义中分离词法规则并写为正则定义式，这一过程让我体会到正则表达式在词法级别模式匹配中够¨ 用但不完美的¨ 特性——它无法表达长度限制（如标识符 ≤ 24 字符）、无法表达上下文依赖（如 25id 中整数与标识符的非法拼接），这些都需要在语义动作中用 Java 代码补充检测。

在编写 JFlex 输入源文件 `oberon.flex` 时，对规则顺序和最长匹配原则的理解至关重要。保留字规则必须置于标识符规则之前才能被优先匹配，检测非法整数拼接（如 25id）的规则必须置于正确整数规则之前。注释处理中%state 状态机制的使用也让我切实体会到词法分析中上¨ 下文的¨ 概念——注释内的字符与注释外的字符需要完全不同的匹配规则。

通过对 JFlex、JLex 和 GNU Flex 三种工具的比较，我理解了词¨ 法分析器生成器这¨ 类工具的共性设计模式（三段式输入结构、正则表达式 → NFA → DFA → 最小化 DFA 的自动机转换流程），也认识到不同工具在目标语言集成和特性支持上的权衡。JFlex 作为 JLex 的后继者，在 Unicode 支持、EOF 处理和性能优化方面的改进充分体现了开源软件演进的活力。